

Домашнее задание: Реализация прямого  
симплекс-метода (Фаза I и Фаза II)

# Введение

Цель этого задания — реализовать полный алгоритм симплекс-метода для решения задач линейного программирования (ЗЛП). Вы реализуете обе фазы: Фазу I для нахождения начального допустимого базисного решения и Фазу II для перехода к оптимальному решению.

Ключевым аспектом этого задания является эффективная работа с базисной матрицей. Вместо того чтобы пересчитывать ее обратную матрицу или LU-разложение на каждой итерации (что вычислительно затратно), вы должны реализовать метод обновления разложения. Это задание познакомит вас с практическими аспектами численной реализации симплекс-метода, которые делают его эффективным на практике.

## 0.1 Формат задачи (входной файл)

Мы решаем ЗЛП в канонической форме:

$$\begin{aligned} &\text{максимизировать} && c^T x \\ &\text{при ограничениях} && Ax \leq b, \\ &&& x \geq 0. \end{aligned}$$

Формат входного текстового файла:

```
n m
c1 c2... cn
a11 a12... a1n b1
a21 a22... a2n b2
...
am1 am2... amn bm
```

- Первая строка:  $n$  (количество переменных) и  $m$  (количество ограничений).
- Вторая строка: вектор целевой функции  $c$  (длиной  $n$ ).
- Следующие  $m$  строк: каждая строка матрицы  $A$ , за которой следует соответствующий элемент  $b_i$ .

Используйте небольшой численный допуск (например,  $10^{-9}$ ) для проверок на ноль или сравнения чисел с плавающей точкой.

## 0.2 Требуемый формат вывода

Ваша программа должна выводить на экран или возвращать:

- **status**  $\in \{\text{"optimal"}, \text{"infeasible"}, \text{"unbounded"}\}$  (статус решения: оптимальное, недопустимое, неограниченное).
- **x\_star** (вектор оптимального решения для прямых переменных  $x$ ; не выводить, если решение не оптимально).

### 0.3 Алгоритм для реализации — Прямой симплекс-метод (Фаза II)

Этот алгоритм начинается с допустимого базиса (который вы получите с помощью Фазы I). На каждой итерации мы работаем с **пересмотренным симплекс-методом**, который избегает явного вычисления всего словаря, оперируя напрямую с базисной матрицей и ее разложением.

Пусть  $\mathcal{B}$  — множество индексов базисных переменных, а  $\mathcal{N}$  — небазисных ( $|\mathcal{B}| = m$ ). Матрица  $A$  и вектор  $c$  разбиваются соответствующим образом:  $A =$  и  $c^T =$ .

**Ключевые вычисления на каждой итерации:**

- $\bar{b} = B^{-1}b$  (текущие значения базисных переменных).
- $y = (B^T)^{-1}c_{\mathcal{B}}$  (вектор симплекс-множителей или двойственных переменных).
- $r_{\mathcal{N}} = c_{\mathcal{N}} - N^T y$  (вектор приведенных стоимостей для небазисных переменных).
- $z_0 = c_{\mathcal{B}}^T \bar{b}$  (текущее значение целевой функции).

Эти величины соответствуют словарю:

$$\begin{aligned} z &= z_0 + r_{\mathcal{N}}^T x_{\mathcal{N}} \\ x_{\mathcal{B}} &= \bar{b} - (B^{-1}N)x_{\mathcal{N}} \end{aligned} \quad (x_{\mathcal{N}} \geq 0, x_{\mathcal{B}} \geq 0).$$

**Итерация симплекс-метода:**

1. **Проверка на оптимальность:** Если  $r_j \leq 0$  для всех  $j \in \mathcal{N}$ , остановиться. Текущее решение оптимально. Статус: "optimal". Вывести  $x_{\mathcal{B}} = \bar{b}$ ,  $x_{\mathcal{N}} = 0$ .
2. **Выбор входящей переменной:** Выбрать индекс  $j \in \mathcal{N}$ , для которого  $r_j > 0$ . Это переменная  $x_j$ , которая войдет в базис. (Можно использовать правило наибольшего коэффициента или правило Блэнда для предотвращения заикливания).
3. **Вычисление направления:** Вычислить направляющий столбец  $d = B^{-1}a_j$ , где  $a_j$  — это столбец матрицы  $A$ , соответствующий переменной  $x_j$ .
4. **Проверка на неограниченность:** Если  $d_i \leq 0$  для всех  $i = 1, \dots, m$ , то задача неограничена. Статус: "unbounded".
5. **Критерий минимального отношения (выбор выходящей переменной):** Найти шаг  $\theta^*$ :

$$\theta^* = \min_{i: d_i > 0} \frac{\bar{b}_i}{d_i}.$$

Пусть  $p$  — это индекс строки (среди базисных переменных), где достигается минимум. Переменная, соответствующая этой строке, покидает базис.

6. **Опорная операция (пивот):** Обновить базис. Индекс  $j$  (входящая переменная) добавляется в  $\mathcal{B}$ , а индекс базисной переменной, соответствующей строке  $p$ , удаляется из  $\mathcal{B}$  и добавляется в  $\mathcal{N}$ . Обновить базисную матрицу  $B$  и ее разложение (см. раздел 5).
7. Повторить с шага 1.

## 0.4 Фаза I (для нахождения начального допустимого базиса)

Сначала преобразуем задачу  $Ax \leq b$  в стандартную форму с уравнениями, введя балансовые (слабые) переменные  $s \geq 0$ :  $Ax + s = b$ . Начальный базис, состоящий из переменных  $s$ , является допустимым только если все  $b_i \geq 0$ .

Если некоторые  $b_i < 0$ , начальное решение  $x = 0, s = b$  недопустимо. Для нахождения допустимого решения решается **вспомогательная задача**. Вводится одна искусственная переменная  $x_0 \geq 0$ :

$$\begin{aligned} & \text{максимизировать} && (-x_0) \\ & \text{при ограничениях} && Ax + s - x_0 \mathbf{1} = b, \\ & && x, s, x_0 \geq 0. \end{aligned}$$

где  $\mathbf{1}$  — это вектор из единиц.

### Шаги Фазы I:

1. Если все  $b_i \geq 0$ , начальный базис из балансовых переменных  $s$  является допустимым. Переходите сразу к Фазе II.
2. В противном случае, сформируйте вспомогательную задачу. Начальный словарь для нее недопустим:

$$\begin{aligned} z_{P1} &= -x_0 \\ s &= b + x_0 \mathbf{1} - Ax \end{aligned}$$

3. Выполните **одну специальную опорную операцию**, чтобы сделать словарь допустимым. Выберите строку  $p$  с наиболее отрицательным  $b_p$ . Введите в базис  $x_0$  и выведите из базиса  $s_p$ . Это создаст допустимый базис для Фазы I.
4. Решите вспомогательную задачу, используя алгоритм Фазы II, максимизируя  $(-x_0)$ .

### 5. Анализ результата Фазы I:

- Если оптимальное значение равно 0 (т.е.  $x_0 = 0$ ), то найден допустимый базис для исходной задачи. Исключите  $x_0$  из базиса (если он там остался) и из всех уравнений, восстановите исходную целевую функцию и переходите к Фазе II.
- Если оптимальное значение строго меньше 0 (т.е.  $x_0 > 0$ ), то исходная задача не имеет допустимых решений. Статус: "infeasible".

## 0.5 Эффективное обновление базиса

На каждой итерации симплекс-метода меняется только один столбец базисной матрицы  $B$ . Полный пересчет  $B^{-1}$  или  $LU$ -разложения ( $O(m^3)$ ) неэффективен. Вместо этого следует обновлять разложение за  $O(m^2)$ .

## (А) Обновление LU-разложения (рекомендуемый подход)

Этот метод, известный как метод Бартельса-Голуба, является стандартом в современных решателях. Идея состоит в том, чтобы поддерживать  $LU$ -разложение базисной матрицы  $B = LU$  (для простоты опустим матрицы перестановок  $P$  и  $Q$ ) и обновлять его на каждой итерации.

Пусть на текущей итерации столбец  $a_p$  (который был  $p$ -м столбцом в  $B$ ) заменяется на столбец  $a_j$  (из  $A$ ). Новая базисная матрица  $B_{new}$  отличается от  $B$  только в  $p$ -м столбце.

### Алгоритм обновления $LU$ -разложения:

1. **Исходные данные:** Матрицы  $L$  и  $U$ , где  $B = LU$ .
2. **Вычисление "шипа":** Нам нужен столбец  $d = B^{-1}a_j$ . Мы уже вычисляем его на шаге 3 Фазы II. Для этого решаем две треугольные системы:
  - Решить  $Lv = a_j$  относительно  $v$  (прямая подстановка).
  - Решить  $Ud = v$  относительно  $d$  (обратная подстановка).
3. **Формирование промежуточной матрицы:** Создадим матрицу  $H = L^{-1}B_{new}$ . Поскольку  $B_{new}$  совпадает с  $B$  во всех столбцах, кроме  $p$ -го, то  $H$  будет совпадать с  $U$  во всех столбцах, кроме  $p$ -го. В  $p$ -м столбце  $H$  будет стоять вектор  $v = L^{-1}a_j$ , который мы нашли на предыдущем шаге.

$$H = [u_1, \dots, u_{p-1}, v, u_{p+1}, \dots, u_m]$$

Матрица  $H$  является верхнетреугольной, за исключением "шипа" (ненулевых поддиагональных элементов) в столбце  $p$ . Такая матрица называется **верхней хессенберговой**.

4. **Восстановление треугольной формы:** Наша цель — преобразовать  $H$  обратно в верхнетреугольную матрицу  $U_{new}$  с помощью элементарных преобразований строк (например, исключения Гаусса), которые обнуляют поддиагональные элементы в столбце  $p$ .
  - Последовательно, для  $i$  от  $m-1$  до  $p$ , используйте строку  $i+1$  для обнуления элемента  $h_{i+1,p}$  в строке  $i$ . Это может потребовать перестановки строк для численной устойчивости (аналогично частичному пивотированию).
  - Каждое такое преобразование можно представить как умножение слева на элементарную матрицу  $G_i$ .

В результате мы получим:

$$U_{new} = G_p \cdots G_{m-1} H$$

5. **Обновление факторов:** Новое разложение имеет вид  $B_{new} = L_{new}U_{new}$ , где:

$$L_{new} = L(G_{m-1}^{-1} \cdots G_p^{-1})$$

На практике матрицы  $G_i^{-1}$  не перемножаются с  $L$ , а хранятся в виде произведения (product form), чтобы сохранить разреженность.

Этот процесс требует  $O(m^2)$  операций и является численно более стабильным, чем обновление явной обратной матрицы.

## (В) Обновление по формуле Шермана-Моррисона (проще для реализации)

Этот метод обновляет явную обратную матрицу  $B^{-1}$ . Он прост в кодировании, но может быть численно нестабильным. Если столбец  $b_p$  в  $B$  заменяется на  $a_j$ , то новую матрицу можно записать как  $B' = B + (a_j - b_p)e_p^T$ . Положим  $u = a_j - b_p$  и  $v = e_p$ . Тогда формула Шермана-Моррисона для обратной матрицы:

$$(B')^{-1} = B^{-1} - \frac{(B^{-1}u)(v^T B^{-1})}{1 + v^T B^{-1}u}$$

Знаменатель упрощается до  $p$ -го элемента вектора  $d = B^{-1}a_j$ , то есть  $d_p$ . Это позволяет обновить  $B^{-1}$  за  $O(m^2)$  операций.

## 0.6 Пример задачи (3 переменных, 4 ограничения)

Задача в канонической форме:

$$\begin{array}{ll}\text{максимизировать} & 3x_1 + 2x_2 + x_3 \\ \text{при ограничениях} & x_1 + x_2 + x_3 \leq 30, \\ & 2x_1 + x_2 \leq 40, \\ & x_1 + 3x_2 + x_3 \leq 60, \\ & x_3 \leq 10, \\ & x \geq 0.\end{array}$$

Текстовый файл (`example1.txt`):

```
3 4
3 2 1
1 1 1 30
2 1 0 40
1 3 1 60
0 0 1 10
```

## 0.7 Что необходимо сдать

1. Ваш исходный код (Python или MATLAB/Octave).
2. Краткий файл `README`, описывающий ваш дизайн (структура данных, правило разрешения ничьих (tie-breaking), численные допуски).
3. Лог запуска вашей программы на файле `example1.txt`, показывающий итоговый статус и вектор решения  $x^*$ .

## Рекомендуемая литература

1. D. G. Luenberger & Y. Ye, *Linear and Nonlinear Programming*, 4th ed., Springer — пересмотренный симплекс-метод, Фазы I/II и численные аспекты.

2. R. H. Bartels & G. H. Golub, “The simplex method of linear programming using LU decomposition,” *Commun. ACM*, 1969 — одна из первых работ по ревизованному симплекс-методу на основе LU-разложения.
3. J. J. H. Forrest & J. A. Tomlin, “Updating triangular factors of the basis...,” *Mathematical Programming*, 1972 — классическая работа по обновлению LU-разложения для разреженных матриц.